

МЕТОДЫ ВЕКТОРИЗАЦИИ ВЫЧИСЛЕНИЙ В ЗАДАЧАХ СУПЕРКОМПЬЮТЕРНОГО МОДЕЛИРОВАНИЯ

А.А. Рыбаков¹

¹ Национальный исследовательский центр «Курчатовский институт», Москва, Россия

Векторизация - низкоуровневая оптимизация программного кода, позволяющая достичь кратного ускорения суперкомпьютерных приложений. Все основные современные микропроцессорные архитектуры поддерживают векторные вычисления, наблюдается тенденция на увеличение размера вектора. В докладе рассмотрены проблемы векторизации программного контекста различного вида с использованием набора инструкций AVX-512. Рассмотренные примеры тестировались на линейке микропроцессоров Intel, начиная с Intel Xeon Phi Knights Langing. Результаты исследований нашли свое отражение в ряде проектов МСЦ РАН, ЦИАМ им. П.И. Баранова и НИЦ «Курчатовский институт».

CALCULATIONS VECTORIZATION METHODS IN PROBLEMS OF SUPERCOMPUTER MODELING

A.A. Rybakov¹

¹ National Research Centre "Kurchatov Institute", Moscow, Russia

Vectorization is a low-level optimization of program code that allows achieving multiple acceleration of supercomputer applications. All major modern microprocessor architectures support vector calculations, and there is a tendency to increase the vector size. The report considers the problems of vectorization of various types of program contexts using the AVX-512 instruction set. The considered examples were tested on a line of Intel microprocessors, starting with Intel Xeon Phi Knights Langing. The research results were reflected in projects of the JSCC RAS, Central Institute of Aviation Motors, and the National Research Center "Kurchatov Institute".

Векторные инструкции представлены во всех современных микропроцессорных архитектурах – x86, ARM, Power, «Эльбрус», LoongArch, Sunway – однако наиболее перспективным является набор векторных инструкций AVX-512, поддерживающий выборочную обработку элементов векторов с помощью векторных масок [1].

В качестве программного контекста, удобного для применения векторизации, можно рассмотреть плоский цикл [2]. Основная идея плоского цикла состоит в объединении нескольких вызовов (в количестве, равном ширине векторизации w) скалярной функции в единую векторную реализацию (рис. 1). Здесь w экземпляров функции fun со скалярными параметрами и скалярным результатом объединены в функцию FUN с векторными параметрами и векторным результатом.

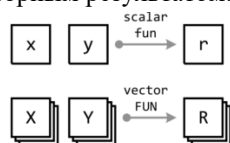


Рис. 1. Схема выполнения скалярной функции fun и аналогичной векторной функции FUN

В качестве плоского цикла рассматривается цикл for , в котором индуктивная переменная i последовательно принимает значения от 0 до $w - 1$ ($\text{for } (\text{int } i = 0; i < w; ++i)$). К плоскому циклу предъявляются следующие требования. На i -ой итерации все обращения к данным на запись имеют вид $a[i]$, а обращения к данным на чтение либо имеют вид $a[i]$, либо являются чтением скаляров. Все массивы данных, обращение к которым на i -ой итерации цикла имеет вид $a[i]$, выровнены в памяти на размер вектора. В цикле отсутствуют межитерационные зависимости. Программный контекст, удовлетворяющий требованиям, предъявляемым к плоским циклам, является удобным для векторизации, а логика работы многих векторных инструкций может быть представлена в виде плоского цикла и записана в предикатной форме. При компоновке программного кода в виде композиции плоских циклов векторизация зачастую может быть применена автоматически оптимизирующим компилятором. При отклонении от требований плоского цикла (квазиплоский цикл) автоматическая векторизация также может быть выполнена, однако ее эффективность при этом

снижается. При отказе от автоматической векторизации, она может быть выполнена с помощью функций-интринсиков [3, 4].

2 Методы векторизации плоских циклов

Для подготовки программного контекста для автоматической векторизации целесообразно использовать хранение обрабатываемых данных в виде «набора массивов» вместо «массива структур», обеспечивая таким образом возможность объединения операций чтения/записи элементов данных на соседних итерациях плоского цикла в широкие векторные команды чтения/записи.

Хотя возможна векторизация плоского цикла с телом практически произвольного вида, основной причиной отказа от автоматической векторизации является наличие команд передачи управления внутри тела плоского цикла (операторы if, switch, break и другие). Для удаления команд передачи управления могут быть применены оптимизации расщепления цикла по условию (для константных условий) или выноса маловероятного региона из цикла (для неконстантных условий) [5].

Универсальной оптимизацией для устранения условий внутри тела плоского цикла является слияние путей исполнения по условию с помощью постановки операций из параллельных линейных участков под соответствующие предикаты (рис. 2).

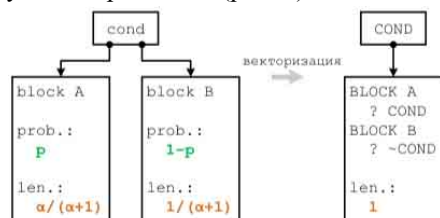


Рис. 2. Слияние двух линейных участков по условию

Эффективность такой оптимизации падает экспоненциально с ростом количества условий внутри тела плоского цикла. Несколько повысить эффективность векторизации для отдельных приложений можно с помощью проверки векторной маски на пустоту перед выполнением векторизованного блока.

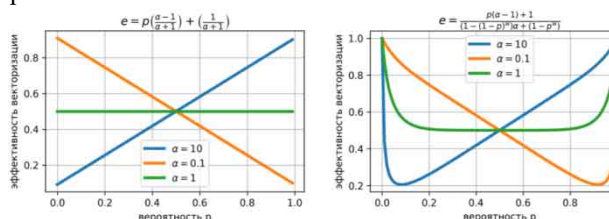


Рис. 3. Зависимость эффективности векторизации от вероятности условия p и разных отношений длин линейных участков $\alpha = 10, 0.1, 1$ при

простом слиянии (слева) и с проверкой векторной маски (справа)

Проверку векторных масок целесообразно выполнять для программного кода задач моделирования физических процессов, для которых характерно большое количество пустых масок.

Для повышения эффективности векторизации необходимо повышать плотность векторных масок. Этого можно добиться с помощью объединения и комбинирования масок соседних векторных блоков [6]. Суть объединения векторных масок заключается в следующем. Если есть два соседних векторных блока $\text{in_data_1} \rightarrow \text{block} \rightarrow \text{out_data_1}$ и $\text{in_data_2} \rightarrow \text{block} \rightarrow \text{out_data_2}$, которые должны выполняться под разными векторными масками mask_1 и mask_2 , и в дополнение к этому для этих масок выполнено условие $(\text{mask_1} \& \text{mask_2}) == 0x0$ (то есть маски не пересекаются), то вычисление этих двух соседних блоков можно объединить. Вместо последовательного выполнения двух векторных блоков можно объединить входные данные in_data_1 и in_data_2 с помощью слияния $\text{in_data} = \text{blend}(\text{mask_1}, \text{in_data_2}, \text{in_data_1})$, после чего выполнить тот же блок вычислений под маской $\text{mask_1} | \text{mask_2}$. Ввиду отсутствия пересечения векторных масок в результирующих выходных данных out_data будут содержаться как необходимые элементы данных out_data_1 , так и необходимые элементы данных out_data_2 . Последним действием, которое нужно выполнить, является извлечение из объединенного результата out_data данных out_data_1 и out_data_2 (рис. 4).

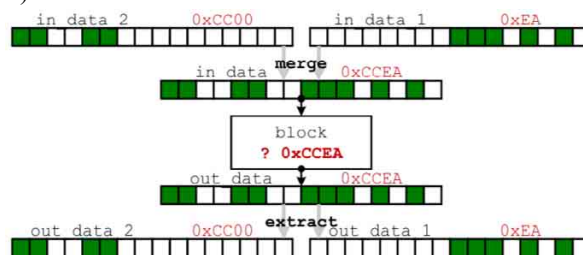


Рис. 4. Схема векторизации с объединением векторных масок

Комбинирование масок позволяет похожим образом объединять векторные блоки, но уже для пересекающихся масок. Оптимизации объединения и комбинирования векторных масок добавляют накладные расходы на подготовку данных и извлечение результата, но сокращают количество векторных блоков.

Особым случаем векторизации плоского цикла является случай, когда тело плоского цикла само содержит вложенные гнезда циклов (внутренние циклы). В этом случае эффективность векторизации

критическим образом зависит от характера внутренних циклов. В наиболее простом случае, когда количество итераций внутренних циклов постоянно, условия выхода из внутренних циклов не меняются при векторизации, что обеспечивает наилучшую эффективность векторизации. Если количество итераций внутренних циклов непостоянно, то сами условия выхода из внутренних циклов необходимо векторизовать (рис. 5).

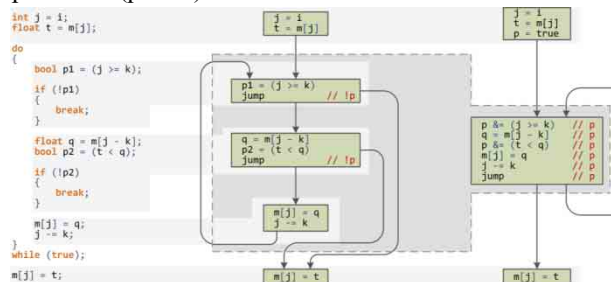


Рис. 5. Схема перевода тела внутреннего цикла в предикатную форму для последующей векторизации

Эффективность векторизации такого программного контекста зависит от того, как меняется условие выхода из внутреннего цикла. Если это условие меняется достаточно медленно (регулярное количество итераций внутреннего цикла), то в векторизованной версии внутреннего цикла маска выполнения итераций обнуляется достаточно быстро. Если же условие выхода меняется непредсказуемым образом (нерегулярное количество итераций внутреннего цикла), то в векторизованной версии внутреннего цикла будет большое количество почти пустых масок, что приводит к сильному падению эффективности векторизации [7].

3 Заключение

По результатам проведенных исследований была собрана статистика по векторизации программного контекста различного вида: операции над матрицами малой размерности, функции из приближенных газодинамических решателей, а также точного римановского решателя, функции обработки граничных условий с помощью метода погруженной границы, функции для работы с геометрическими примитивами, а также примеры циклов с нерегулярным количеством итераций и другие.

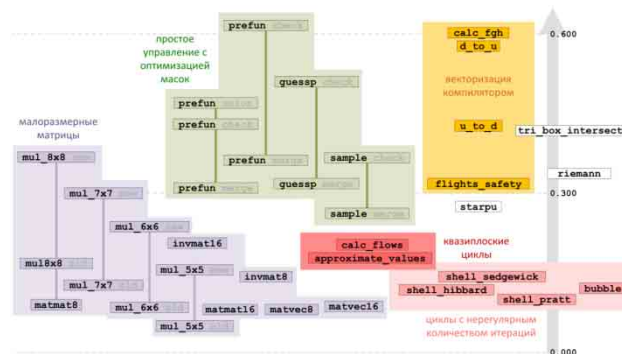


Рис. 6. Карта эффективности векторизации

Для всех рассмотренных функций была замерена эффективность векторизации на микропроцессоре Intel Xeon Phi Knights Landing и результаты собраны на рис. 6.

Из карты можно сделать вывод, что наиболее неудобным программным контекстом для векторизации являются гнезда циклов с нерегулярным количеством итераций, а также квазиплоские циклы, то есть циклы, нарушающие некоторые требования, предъявляемые к плоским циклам (дополнительная косвенность при обращении в память, невыровненность данных и другие). Наиболее удобным контекстом для векторизации являются плоские циклы с простым управлением и применением проверок и объединения масок.

Список использованных источников

- [1] Intel 64 and IA-32 architectures software developer's manual. Combined volumes: 1, 2A, 2B, 2C, 2D, 3A, 3B, 3C, 3D, and 4. // Order number: 325462-087US, march 2025.
- [2] Savin G. I., Shabanov B. M., Rybakov A. A., Shumilin S. S. Vectorization of flat loops of arbitrary structure using instructions AVX-512. // Lobachevskii Journal of Mathematics, 2020, Vol. 41, No. 12, p. 2566-2574.
- [3] Intel Intrinsics Guide, <https://alouettesu.github.io/Intrinsics/> (дата обращения 01.05.2025)
- [4] Shabanov B. M., Rybakov A. A., Shumilin S. S. Vectorization of high-performance scientific calculations using AVX-512 instruction set. // Lobachevskii Journal of Mathematics, 2019, Vol. 40, No. 5, p. 580-598.
- [5] Шабанов Б. М., Рыбаков А. А., Чопорняк А. Д. Оптимизации, применяемые к графу потока управления программы для повышения эффективности векторизации плоских циклов. // Труды НИИСИ РАН, 2021, Т. 11, № 2, с. 11-19.
- [6] Рыбаков А.А. Векторизация циклов с условными операциями с помощью комбинирования векторных масок. // Современные информаци-

онные технологии и ИТ-образование, 2024, Т. 20,
№ 3, с. 520-534.

- [7] Рыбаков А. А., Шумилин С. С. Исследование эффективности векторизации гнезд циклов с нерегулярным числом итераций. // Программные системы: Теория и алгоритмы, 2019, Т. 10, № 4 (43), с. 77-96.